jmatrix

1.1

Generated by Doxygen 1.9.1

1 Hierarchical Index 1

1.1 Class Hierarchy . . . 1

2 Class Index 3

2.1 Class List . . . 3

3 File Index 5

3.1 File List . . . 5

4 Class Documentation 7

4.1 FullMatrix< T > Class Template Reference . . . 7

4.1.1 Detailed Description . . . 8

4.1.2 Constructor & Destructor Documentation . . . 8

4.1.2.1 FullMatrix() [1/7] . . . 8

4.1.2.2 FullMatrix() [2/7] . . . 8

4.1.2.3 FullMatrix() [3/7] . . . 8

4.1.2.4 FullMatrix() [4/7] . . . 9

4.1.2.5 FullMatrix() [5/7] . . . 9

4.1.2.6 FullMatrix() [6/7] . . . 9

4.1.2.7 FullMatrix() [7/7] . . . 10

4.1.2.8 ~FullMatrix() . . . 10

4.1.3 Member Function Documentation . . . 10

4.1.3.1 Get() . . . 10

4.1.3.2 GetFullRow() . . . 11

4.1.3.3 GetMarksOfFullRow() . . . 11

4.1.3.4 GetRow() . . . 12

4.1.3.5 GetUsedMemoryMB() . . . 12

4.1.3.6 operator"!=() . . . 12

4.1.3.7 operator=() . . . 13

4.1.3.8 Resize() . . . 13

4.1.3.9 SelfColNorm() . . . 13

4.1.3.10 SelfRowNorm() . . . 14

4.1.3.11 Set() . . . 14

4.1.3.12 WriteBin() . . . 14

4.1.3.13 WriteCsv() . . . 15

4.2 JMatrix< T > Class Template Reference . . . 15

4.2.1 Detailed Description . . . 16

4.2.2 Constructor & Destructor Documentation . . . 16

4.2.2.1 JMatrix() [1/5] . . . 16

4.2.2.2 JMatrix() [2/5] . . . 17

4.2.2.3 JMatrix() [3/5] . . . 17

4.2.2.4 JMatrix() [4/5] . . . 17

4.2.2.5 JMatrix() [5/5] . . . 18

4.2.3 Member Function Documentation . . . 18
4.2.3.1 GetColNames() . . . 18
4.2.3.2 GetComment() . . . 19
4.2.3.3 GetNCols() . . . 19
4.2.3.4 GetNRows() . . . 19
4.2.3.5 GetRowNames() . . . 19
4.2.3.6 operator"!=() . . . 19
4.2.3.7 operator=() . . . 20
4.2.3.8 Resize() . . . 20
4.2.3.9 SetColNames() . . . 20
4.2.3.10 SetComment() . . . 21
4.2.3.11 SetRowNames() . . . 21
4.2.3.12 WriteBin() . . . 21
4.2.3.13 WriteCsv() . . . 22
4.3 SparseMatrix< T > Class Template Reference . . . 23
4.3.1 Detailed Description . . . 23
4.3.2 Constructor & Destructor Documentation . . . 24
4.3.2.1 SparseMatrix() [1/6] . . . 24
4.3.2.2 SparseMatrix() [2/6] . . . 24
4.3.2.3 SparseMatrix() [3/6] . . . 24
4.3.2.4 SparseMatrix() [4/6] . . . 24
4.3.2.5 SparseMatrix() [5/6] . . . 25
4.3.2.6 SparseMatrix() [6/6] . . . 25
4.3.2.7 ~SparseMatrix() . . . 26
4.3.3 Member Function Documentation . . . 26
4.3.3.1 Get() . . . 26
4.3.3.2 GetMarksOfSparseRow() . . . 26
4.3.3.3 GetRow() . . . 27
4.3.3.4 GetSparseRow() . . . 27
4.3.3.5 GetUsedMemoryMB() . . . 27
4.3.3.6 operator"!=() . . . 28
4.3.3.7 operator=() . . . 28
4.3.3.8 Resize() . . . 28
4.3.3.9 SelfColNorm() . . . 29
4.3.3.10 SelfRowNorm() . . . 29
4.3.3.11 Set() . . . 29
4.3.3.12 SetRow() . . . 30
4.3.3.13 WriteBin() . . . 30
4.3.3.14 WriteCsv() . . . 31
4.4 SymmetricMatrix< T > Class Template Reference . . . 31
4.4.1 Detailed Description . . . 32
4.4.2 Constructor & Destructor Documentation . . . 32

4.4.2.1 SymmetricMatrix() [1/6] . . . 32
4.4.2.2 SymmetricMatrix() [2/6] . . . 32
4.4.2.3 SymmetricMatrix() [3/6] . . . 32
4.4.2.4 SymmetricMatrix() [4/6] . . . 33
4.4.2.5 SymmetricMatrix() [5/6] . . . 33
4.4.2.6 SymmetricMatrix() [6/6] . . . 33
4.4.2.7 ~SymmetricMatrix() . . . 34
4.4.3 Member Function Documentation . . . 34
4.4.3.1 Get() . . . 34
4.4.3.2 GetRowSum() . . . 34
4.4.3.3 GetUsedMemoryMB() . . . 35
4.4.3.4 operator=() . . . 35
4.4.3.5 Resize() . . . 35
4.4.3.6 Set() . . . 36
4.4.3.7 TestDistDisMat() . . . 36
4.4.3.8 WriteBin() . . . 36
4.4.3.9 WriteCsv() . . . 37

5 File Documentation **39**
5.1 src/examples/jmat.cpp File Reference . . . 39
5.1.1 Detailed Description . . . 39
5.1.2 Function Documentation . . . 39
5.1.2.1 main() . . . 40
5.2 src/headers/apitocommands.h File Reference . . . 42
5.2.1 Function Documentation . . . 42
5.2.1.1 JCsvDump() . . . 42
5.2.1.2 JCsvToJMat() . . . 43
5.2.1.3 JGetColNames() . . . 43
5.2.1.4 JGetNameCol() . . . 43
5.2.1.5 JGetNameRow() . . . 44
5.2.1.6 JGetNamesCol() . . . 44
5.2.1.7 JGetNamesRow() . . . 44
5.2.1.8 JGetNumCol() . . . 45
5.2.1.9 JGetNumRow() . . . 45
5.2.1.10 JGetNumsCol() . . . 46
5.2.1.11 JGetNumsRow() . . . 46
5.2.1.12 JGetRowNames() . . . 46
5.2.1.13 JGetSubDiag() . . . 47
5.2.1.14 JMatInfo() . . . 47
5.2.1.15 JSetColNames() . . . 47
5.2.1.16 JSetRowColNames() . . . 48
5.2.1.17 JSetRowNames() . . . 48

5.3 src/headers/debugpar.h File Reference . . . 48
5.3.1 Function Documentation . . . 49
5.3.1.1 JMatrixSetDebug() . . . 49
5.3.1.2 JMatrixStop() . . . 49
5.3.1.3 JMatrixWarning() . . . 49
5.3.2 Variable Documentation . . . 50
5.3.2.1 DEBJM . . . 50
5.3.2.2 NODEBUG . . . 50
5.4 src/headers/fullmatrix.h File Reference . . . 50
5.5 src/headers/indextype.h File Reference . . . 50
5.5.1 Typedef Documentation . . . 50
5.5.1.1 indextype . . . 51
5.6 src/headers/jmatrix.h File Reference . . . 51
5.6.1 Function Documentation . . . 53
5.6.1.1 DataTypeName() . . . 53
5.6.1.2 GetFileSize() . . . 54
5.6.1.3 MatrixTypeName() . . . 54
5.6.1.4 MetadataInfo() . . . 55
5.6.1.5 PositionsInFile() . . . 55
5.6.1.6 SizeOfType() . . . 55
5.6.1.7 ThisMachineEndianness() . . . 56
5.6.2 Variable Documentation . . . 56
5.6.2.1 BIGEND . . . 56
5.6.2.2 BLOCK_MARK . . . 56
5.6.2.3 BLOCKSEP . . . 56
5.6.2.4 BLOCKSEP_LEN . . . 56
5.6.2.5 COL_NAMES . . . 57
5.6.2.6 COMMENT . . . 57
5.6.2.7 COMMENT_SIZE . . . 57
5.6.2.8 DTYPE . . . 57
5.6.2.9 ERROR_READING_COL_NAMES . . . 57
5.6.2.10 ERROR_READING_ROW_NAMES . . . 57
5.6.2.11 ERROR_READING_SEP_MARK . . . 57
5.6.2.12 ERROR_READING_STRINGS . . . 58
5.6.2.13 FTYPE . . . 58
5.6.2.14 HEADER_SIZE . . . 58
5.6.2.15 LDTYPE . . . 58
5.6.2.16 LITEND . . . 58
5.6.2.17 MAX_LEN_NAME . . . 58
5.6.2.18 MTYPEFULL . . . 58
5.6.2.19 MTYPENOTYPE . . . 59
5.6.2.20 MTYPESPARSE . . . 59

5.6.2.21 MTYPESYMMETRIC . . . 59
5.6.2.22 NO_METADATA . . . 59
5.6.2.23 NOTYPE . . . 59
5.6.2.24 READ_OK . . . 59
5.6.2.25 ROW_NAMES . . . 59
5.6.2.26 SCTYPE . . . 60
5.6.2.27 SITYPE . . . 60
5.6.2.28 SLLTYPE . . . 60
5.6.2.29 SLTYPE . . . 60
5.6.2.30 SSTYPE . . . 60
5.6.2.31 UCTYPE . . . 60
5.6.2.32 UITYPE . . . 60
5.6.2.33 ULLTYPE . . . 60
5.6.2.34 ULTYPE . . . 61
5.6.2.35 USTYPE . . . 61
5.7 src/headers/matinfo.h File Reference . . . 61
5.7.1 Function Documentation . . . 61
5.7.1.1 JMatInfo() . . . 61
5.7.1.2 MatrixType() [1/5] . . . 62
5.7.1.3 MatrixType() [2/5] . . . 62
5.7.1.4 MatrixType() [3/5] . . . 63
5.7.1.5 MatrixType() [4/5] . . . 63
5.7.1.6 MatrixType() [5/5] . . . 64
5.8 src/headers/matmetadata.h File Reference . . . 64
5.8.1 Function Documentation . . . 65
5.8.1.1 JGetColNames() . . . 65
5.8.1.2 JGetRowNames() . . . 65
5.9 src/headers/memhelper.h File Reference . . . 65
5.9.1 Function Documentation . . . 66
5.9.1.1 GetAvailableMemAndSwap() . . . 66
5.9.1.2 MemoryWarnings() [1/2] . . . 66
5.9.1.3 MemoryWarnings() [2/2] . . . 66
5.10 src/headers/sparsematrix.h File Reference . . . 67
5.11 src/headers/symmetricmatrix.h File Reference . . . 67

Index **69**

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

JMatrix< T > 15
- FullMatrix< T > 7
- SparseMatrix< T > 23
- SymmetricMatrix< T > 31

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

FullMatrix< T > 7
JMatrix< T > 15
SparseMatrix< T > 23
SymmetricMatrix< T > 31

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

src/examples/jmat.cpp 39
src/headers/apitocommands.h 42
src/headers/debugpar.h 48
src/headers/fullmatrix.h 50
src/headers/indextype.h 50
src/headers/jmatrix.h 51
src/headers/matinfo.h 61
src/headers/matmetadata.h 64
src/headers/memhelper.h 65
src/headers/sparsematrix.h 67
src/headers/symmetricmatrix.h 67

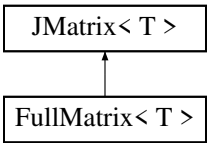
Chapter 4

Class Documentation

4.1 FullMatrix< T > Class Template Reference

```
#include <fullmatrix.h>
```

Inheritance diagram for FullMatrix< T >:



Public Member Functions

- FullMatrix ()
- FullMatrix (indextype nrows, indextype ncols)
- FullMatrix (indextype nrows, indextype ncols, bool warn)
- FullMatrix (const FullMatrix< T > &other)
- FullMatrix (std::string fname)
- FullMatrix (std::string fname, bool warn)
- FullMatrix (std::string fname, unsigned char vtype, char csep)
- void Resize (indextype newnr, indextype newnc)
- ~FullMatrix ()
- FullMatrix< T > & operator= (const FullMatrix< T > &other)
- FullMatrix< T > & operator!= (const FullMatrix< T > &other)
- T Get (indextype r, indextype c)
- void Set (indextype r, indextype c, T v)
- void GetRow (indextype r, T *v)
- void GetFullRow (indextype r, unsigned char *m, unsigned char s, T *v)
- void GetMarksOfFullRow (indextype r, unsigned char *m, unsigned char s)
- void SelfRowNorm (std::string ctype)
- void SelfColNorm (std::string ctype)
- void WriteCsv (std::string fname, char csep=',', bool withquotes=false)
- void WriteBin (std::string fname)
- float GetUsedMemoryMB ()

Additional Inherited Members

4.1.1 Detailed Description

template<typename T>
class FullMatrix< T >

@FullMatrix class to hold full matrices (all space booked in memory)

4.1.2 Constructor & Destructor Documentation

4.1.2.1 FullMatrix() [1/7]

```
template<typename T >
FullMatrix< T >::FullMatrix
```

Default constructor

4.1.2.2 FullMatrix() [2/7]

```
template<typename T >
FullMatrix< T >::FullMatrix (
            indextype nrows,
            indextype ncols )
```

Constructor with number of rows and columns

Parameters

in	*nrows*	Number of rows
in	*ncols*	Number of columns

4.1.2.3 FullMatrix() [3/7]

```
template<typename T >
FullMatrix< T >::FullMatrix (
            indextype nrows,
            indextype ncols,
            bool warn )
```

Constructor with number of rows and columns which gives memory warnings

Parameters

in	*nrows*	Number of rows
in	*ncols*	Number of columns
in	*warn*	Give memory warnings The idea is to use this form of constructor with warn=true, which internally calls MemoryWarnings, if one suspects the matrix to be constructed might provoke memory problems.

4.1.2.4 FullMatrix() [4/7]

```
template<typename T >
FullMatrix< T >::FullMatrix (
            const FullMatrix< T > & other )
```

Copy constructor

Parameters

in	*other*	Reference to the Matrix to be copied

4.1.2.5 FullMatrix() [5/7]

```
template<typename T >
FullMatrix< T >::FullMatrix (
            std::string fname )
```

Constructor to fill the matrix contents from a binary file
Binary file header as explained in the documentation to JMatrix::WriteBin

Parameters

in	*fname*	The name of the file to read

4.1.2.6 FullMatrix() [6/7]

```
template<typename T >
FullMatrix< T >::FullMatrix (
            std::string fname,
            bool warn )
```

Constructor to fill the matrix content from a binary file giving memory warnings
Binary file header as explained in the documentation to JMatrix::WriteBin

Parameters

in	*fname*	The name of the file to read
in	*warn*	Give memory warnings

4.1.2.7 FullMatrix() [7/7]

```
template<typename T >
FullMatrix< T >::FullMatrix (
            std::string fname,
            unsigned char vtype,
            char csep )
```

Constructor to fill the matrix content from a csv file
First line is supposed to have the field names which become the column names
First column of each line is supposed to have the field name, which becomes that row name
The passed character is the expected field sepparator (usually, comma or tab)

Parameters

in	*fname*	The name of the csv file to read
in	*vtype*	The data type to be stored (see constants at jmatrix.h)
in	*csep*	The character used as field sepparator

4.1.2.8 ~FullMatrix()

```
template<typename T >
FullMatrix< T >::~FullMatrix
```

Destructor

4.1.3 Member Function Documentation

4.1.3.1 Get()

```
template<typename T >
T FullMatrix< T >::Get (
            indextype r,
            indextype c )  [inline]
```

Function to get acess to an element

Parameters

in	*r*	The row to access
in	*c*	The columns to access

Returns

value at (r,c) of matrix with type T

4.1.3.2 GetFullRow()

```
template<typename T >
void FullMatrix< T >::GetFullRow (
            indextype r,
            unsigned char * m,
            unsigned char s,
            T * v )
```

Function to get a full row as a pointer to the content type plus a pointer to an array of marks. The content will have the values. The array of marks will be changed OR'ing the passed mark to each place where there is a value. The pointers to the values and marks are not returned since that way they do not need to be booked. They are passed as parameters and both are supposed to be properly allocated. This strange way of storing data has been choosen because it will be specially suitable to calculate distance between vectors

Parameters

in	*r*	The row to get
out	$*m$	Pointer to the marks
in	*s*	The value to be OR'ed to each place at the mark array
out	$*v$	Pointer to the values

4.1.3.3 GetMarksOfFullRow()

```
template<typename T >
void FullMatrix< T >::GetMarksOfFullRow (
            indextype r,
            unsigned char * m,
            unsigned char s )
```

Function to get from a full row a pointer to an array of marks signalling where the non-zero elements are. The array of marks will be changed OR'ing the passed mark to each place where there is a value. The pointer to the marks is not returned since that way it do not need to be booked. It is passed as a parameter and is supposed to be properly allocated.

Parameters

in	*r*	The row to get
out	$*m$	Pointer to the marks
in	*s*	The value to be OR'ed to each place at the mark array

4.1.3.4 GetRow()

```
template<typename T >
void FullMatrix< T >::GetRow (
            indextype r,
            T * v )
```

Function to get a row as a pointer to the content type. The pointer is not returned since that way it does not need to be booked. The pointer to hold result is passed as parameter and it is supposed to be properly allocated.

Parameters

in	*r*	The row to get
out	**v*	Pointer to the result

4.1.3.5 GetUsedMemoryMB()

```
template<typename T >
float FullMatrix< T >::GetUsedMemoryMB
```

Function to get memory in MB used by this full matrix

Returns

The amount of memory in MB

4.1.3.6 operator"!=()

```
template<typename T >
FullMatrix< T > & FullMatrix< T >::operator!= (
            const FullMatrix< T > & other )
```

Transpose-assignment

Parameters

in	*other*	Reference to the Matrix to be assigned

Returns

Reference to the newly created Matrix, which is the transpose of the passed one

4.1.3.7 operator=()

```
template<typename T >
FullMatrix< T > & FullMatrix< T >::operator= (
            const FullMatrix< T > & other )
```

Assignment operator

Parameters

in	*other*	Reference to the Matrix to be assigned

Returns

Reference to the newly created Matrix

4.1.3.8 Resize()

```
template<typename T >
void FullMatrix< T >::Resize (
            indextype newnr,
            indextype newnc )
```

Function to resize the matrix
WARNING: previous content, if any, IS LOST (to be reviewed)

Parameters

in	*newnr*	New number of rows
in	*newnc*	New number of cols

4.1.3.9 SelfColNorm()

```
template<typename T >
void FullMatrix< T >::SelfColNorm (
            std::string ctype )
```

Function to alter the internal values of the matrix so that each column is normalized according to the requested normalization type The purpose of this function can be achieved with a loop using Set and Get, but using the internal structure makes the task much faster.
Normally, this function will not be used outside the context of bioinformatics where these normalizations are standard

Parameters

in	*ctype*	The requested type of normalization: rawn, log1 or log1n

4.1.3.10 SelfRowNorm()

```
template<typename T >
void FullMatrix< T >::SelfRowNorm (
            std::string ctype )
```

Function to alter the internal values of the matrix so that each row is normalized according to the requested normalization type The purpose of this function can be achieved with a loop using Set and Get, but using the internal structure makes the task much faster.
Normally, this function will not be used outside the context of bioinformatics where these normalizations are standard

Parameters

in	*ctype*	The requested type of normalization: rawn, log1 or log1n

4.1.3.11 Set()

```
template<typename T >
void FullMatrix< T >::Set (
            indextype r,
            indextype c,
            T v ) [inline]
```

Function to set an element

Parameters

in	*r*	The row to access
in	*c*	The column to access
in	*v*	The value to be set (of type T)

4.1.3.12 WriteBin()

```
template<typename T >
void FullMatrix< T >::WriteBin (
            std::string fname )
```

Function to write the matrix content to a binary file See format at documentation of JMatrix::WriteBin

Parameters

in	*fname*	The name of the file to write

4.1.3.13 WriteCsv()

```
template<typename T >
void FullMatrix< T >::WriteCsv (
            std::string fname,
            char csep = ',',
            bool withquotes = false )
```

Function to write the matrix content to a CSV file

Parameters

in	*fname*	The name of the file to write
in	*csep*	The separator character between fields (default: , (comma))
in	*withquotes*	Boolean value to indicate if field names in .csv must be written surrounded by quotes.

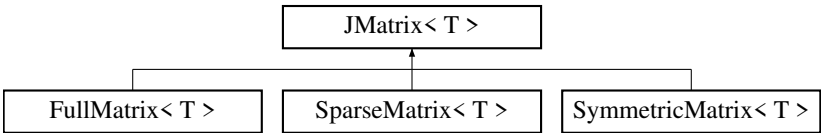
The documentation for this class was generated from the following files:

- src/headers/fullmatrix.h
- src/library/fullmatrix.cpp

4.2 JMatrix< T > Class Template Reference

```
#include <jmatrix.h>
```

Inheritance diagram for JMatrix< T >:



Public Member Functions

- JMatrix (unsigned char mtype)
- JMatrix (unsigned char mtype, indextype nrows, indextype ncols)
- JMatrix (std::string fname, unsigned char mtype)
- JMatrix (std::string fname, unsigned char mtype, unsigned char vtype, char csep)
- JMatrix (const JMatrix< T > &other)
- JMatrix< T > & operator= (const JMatrix< T > &other)
- JMatrix< T > & operator!= (const JMatrix< T > &other)
- indextype GetNRows ()
- indextype GetNCols ()
- void Resize (indextype newnr, indextype newnc)
- std::vector< std::string > GetColNames ()
- std::vector< std::string > GetRowNames ()

- void SetColNames (std::vector< std::string > cnames)
- void SetRowNames (std::vector< std::string > rnames)
- std::string GetComment ()
- void SetComment (std::string cm)
- void WriteCsv (std::string fname, char csep=',', bool withquotes=false)
- void WriteBin (std::string fname, unsigned char mtype)

Protected Member Functions

- unsigned char **TypeNameToId** ()
- bool **ProcessDataLineCsv** (std::string line, char csep, T *rowofdata)
- int **ReadMetadata** ()
- void **WriteMetadata** ()
- void **SetDataType** (unsigned char dtype)

Protected Attributes

- indextype **nr**
- indextype **nc**
- unsigned char **jctype**
- std::ifstream **ifile**
- std::ofstream **ofile**
- std::vector< std::string > **rownames**
- std::vector< std::string > **colnames**
- char **comment** [COMMENT_SIZE]

4.2.1 Detailed Description

template<typename T>
class JMatrix< T >

@JMatrix Wrapper class for all types of matrices. It is meant to hold some basic operations common to all of them Even the instances of this class may now hold real data (the metadata, row and column names), they don't do until an "authentic" matrix is constructed.

4.2.2 Constructor & Destructor Documentation

4.2.2.1 JMatrix() [1/5]

```
template<typename T >
JMatrix< T >::JMatrix (
            unsigned char mtype )
```

Default constructor

Parameters

in	*mtype*	The matrix type (see constants at jmatrix.h)

4.2.2.2 JMatrix() [2/5]

```
template<typename T >
JMatrix< T >::JMatrix (
            unsigned char mtype,
            indextype nrows,
            indextype ncols )
```

Constructor with number of rows and columns

Parameters

in	*mtype*	The matrix type (see constants at jmatrix.h)
in	*nrows*	Number of rows
in	*ncols*	Number of columns

4.2.2.3 JMatrix() [3/5]

```
template<typename T >
JMatrix< T >::JMatrix (
            std::string fname,
            unsigned char mtype )
```

Constructor to fill the matrix content from a binary file

Binary file header as explained in the documentation to WriteBin

TODO PRELIMINARY VERSION. ASSUMES SAME ENDIANESS FOR WRITER AND READER MACHINE

Parameters

in	*fname*	The name of the file to read
in	*mtype*	The matrix type (see constants at jmatrix.h)

4.2.2.4 JMatrix() [4/5]

```
template<typename T >
JMatrix< T >::JMatrix (
```

```
std::string fname,
unsigned char mtype,
unsigned char vtype,
char csep )
```

Constructor to fill the matrix content from a csv text file

Parameters

in	*fname*	The name of the file to read
in	*mtype*	The matrix type (see constants at jmatrix.h)
in	*vtype*	The data type to be contained in the matrix
in	*csep*	The character expected to be the sepparator

4.2.2.5 JMatrix() [5/5]

```
template<typename T >
JMatrix< T >::JMatrix (
            const JMatrix< T > & other )
```

Copy constructor

Parameters

in	*other*	Reference to the JMatrix to be copied

4.2.3 Member Function Documentation

4.2.3.1 GetColNames()

```
template<typename T >
std::vector< std::string > JMatrix< T >::GetColNames
```

Function to get the matrix column names, if present

Returns

: The vector of strings with the col names. Empty vector if not present

4.2.3.2 GetComment()

```
template<typename T >
std::string JMatrix< T >::GetComment
```

Function to get a string with the matrix comment, if any (or the empty string otherwise).

Returns

: A string with the content of the comment area.

4.2.3.3 GetNCols()

```
template<typename T >
indextype JMatrix< T >::GetNCols ( ) [inline]
```

Function to get number of columns

Returns

Number of columns of the matrix

4.2.3.4 GetNRows()

```
template<typename T >
indextype JMatrix< T >::GetNRows ( ) [inline]
```

Function to get number of rows

Returns

Number of rows of the matrix

4.2.3.5 GetRowNames()

```
template<typename T >
std::vector< std::string > JMatrix< T >::GetRowNames
```

Function to get the matrix row names, if present

Returns

: The vector of strings with the row names. Empty vector if not present

4.2.3.6 operator"!=()

```
template<typename T >
JMatrix< T > & JMatrix< T >::operator!= (
            const JMatrix< T > & other )
```

Transpose-assignment

Parameters

in	*other*	Reference to the JMatrix to be assigned

Returns

Reference to the newly created Matrix, which is the transpose of the passed one

4.2.3.7 operator=()

```
template<typename T >
JMatrix< T > & JMatrix< T >::operator= (
            const JMatrix< T > & other )
```

Assignment operator

Parameters

in	*other*	Reference to the JMatrix to be assigned

Returns

Reference to the newly created JMatrix

4.2.3.8 Resize()

```
template<typename T >
void JMatrix< T >::Resize (
            indextype newnr,
            indextype newnc )
```

Function to resize the matrix WARNING: previous content, if any, IS LOST (TODO: to be reviewed)

Parameters

in	*newnr*	New number of rows
in	*newnc*	New number of cols

4.2.3.9 SetColNames()

```
template<typename T >
```

```
void JMatrix< T >::SetColNames (
            std::vector< std::string > cnames )
```

Function to set the matrix column names.

Parameters

in	*cnames*	The std::vector of strings with the column names.

4.2.3.10 SetComment()

```
template<typename T >
void JMatrix< T >::SetComment (
            std::string cm )
```

Function to set the matrix comment

Parameters

in	*cm*	A std::string with the comment to be stored.

4.2.3.11 SetRowNames()

```
template<typename T >
void JMatrix< T >::SetRowNames (
            std::vector< std::string > rnames )
```

Function to set the matrix row names.

Parameters

in	*rnames*	The std::vector of strings with the row names.

4.2.3.12 WriteBin()

```
template<typename T >
void JMatrix< T >::WriteBin (
            std::string fname,
            unsigned char mtype )
```

Function to write the matrix content to a binary file

The binary header will contain:

- unsigned char t: matrix type (normal, sparse, symmetric). Other types might be added later
- unsigned char dt: the data type of the matrix elements, which is one of unsigned/signed char, unsigned signed short, unsigned/signed long, unsigned/signed longlong, float, double, longdouble in its lower 4 bits and the endianess in its upper 4 bits (big or little).
- indextype nr: number of rows
- indextype nc: number of columns
- unsigned char mdinfo: information on the presence/absence of metadata, currently row and/or column names and comment.

This means that the size of the header is 2+2∗sizeof(indextype)+1+empty_space. We have fixed the empty space so that total header size be 128 bytes.

After the header the binary file contains the matrix raw data, by rows. Internal representation is different according to the matrix type (full, sparse or symmetric).

- After the raw data come the row and/or column names (if any)
- Then, the row names, as character arrays separated by null character (0x00)
- Then, a separation mark (the succesion of bytes 0xFF 0x45 0x42 0xFF). 0x45 and 0x42 are ASCII characters EB, for End Block
- Then, the column names, as character arrays separated by null character (0x00)
- Then, a separation mark, as above
- After the row/column names comes the comment (if any)
- Then, a separation mark, as above

HEADER AND DATA ARE ALWAYS WRITTEN IN THE ENDIANESS OF THE MACHINE WHICH EXECUTES THIS CODE Nevertheless, other machines will know about it, since it is declared in the first byte of the written file.

Parameters

in	*string*	fname: The name of the file to write
in	*unsigned*	char mtype: The type identifier

4.2.3.13 WriteCsv()

```
template<typename T >
void JMatrix< T >::WriteCsv (
            std::string fname,
            char csep = ',',
            bool withquotes = false )
```

Function to write the matrix content to a CSV file

Parameters

in	*fname*	The name of the file to write
in	*csep*	The separator character between fields (default: , (comma))
in	*withquotes*	boolean value to indicate if field names in .csv must be written surrounded by quotes or not

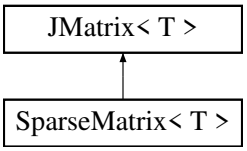
The documentation for this class was generated from the following files:

- src/headers/jmatrix.h
- src/library/jmatrix.cpp

4.3 SparseMatrix< T > Class Template Reference

```
#include <sparsematrix.h>
```

Inheritance diagram for SparseMatrix< T >:



Public Member Functions

- SparseMatrix ()
- SparseMatrix (indextype nrows, indextype ncols)
- SparseMatrix (std::string fname)
- SparseMatrix (std::string fname, TrMark)
- SparseMatrix (std::string fname, unsigned char vtype, char csep)
- void Resize (indextype newnr, indextype newnc)
- SparseMatrix (const SparseMatrix &other)
- ~SparseMatrix ()
- SparseMatrix< T > & operator= (const SparseMatrix< T > &other)
- SparseMatrix< T > & operator!= (const SparseMatrix< T > &other)
- T Get (indextype r, indextype c) const
- void Set (indextype r, indextype c, T v)
- void SetRow (indextype r, std::vector< indextype > vc, std::vector< T > v)
- void GetRow (indextype r, T *v)
- void GetSparseRow (indextype r, unsigned char *m, unsigned char s, T *v)
- void GetMarksOfSparseRow (indextype r, unsigned char *m, unsigned char s)
- void SelfRowNorm (std::string ctype)
- void SelfColNorm (std::string ctype)
- void WriteCsv (std::string fname, char csep=',', bool withquotes=false)
- void WriteBin (std::string fname)
- float GetUsedMemoryMB ()

Additional Inherited Members

4.3.1 Detailed Description

template<typename T>
class SparseMatrix< T >

@SparseMatrix Class to hold arbitrarily big sparse matrices. Elements are stored with column index + value in a vector associated to each row.
Time to set and get elements are of order O(log_2(Nc)) being Nc the number of columns.
Space is O(N*(sizeof(element)+sizeof(index)), being element the type of the matrix contents and index that of the matrix index (which is currently unsigned int).

4.3.2 Constructor & Destructor Documentation

4.3.2.1 SparseMatrix() [1/6]

```
template<typename T >
SparseMatrix< T >::SparseMatrix
```

Default constructor

4.3.2.2 SparseMatrix() [2/6]

```
template<typename T >
SparseMatrix< T >::SparseMatrix (
            indextype nrows,
            indextype ncols )
```

Constructor with number of rows and columns

Parameters

in	*nrows*	Number of rows
in	*ncols*	Number of columns

4.3.2.3 SparseMatrix() [3/6]

```
template<typename T >
SparseMatrix< T >::SparseMatrix (
            std::string fname )
```

Constructor to fill the matrix content from a binary file

Binary file header as explained in the documetation to WriteBin

PRELIMINARY VERSION. ASSUMES SAME ENDIANESS FOR WRITER AND READER MACHINE

Parameters

in	*fname*	The name of the file to read

4.3.2.4 SparseMatrix() [4/6]

```
template<typename T >
SparseMatrix< T >::SparseMatrix (
```

```
            std::string fname,
            TrMark  )
```

Constructor to fill the matrix content from the transpose of the matrix contained in a binary file

Binary file header as explained in the documentation to WriteBin

PRELIMINARY VERSION. ASSUMES SAME ENDIANESS FOR WRITER AND READER MACHINE

Parameters

in	*fname*	The name of the file to read

4.3.2.5 SparseMatrix() [5/6]

```
template<typename T >
SparseMatrix< T >::SparseMatrix (
            std::string fname,
            unsigned char vtype,
            char csep )
```

Constructor to fill the matrix content from a csv file First line is supposed to have the field names and is ignored. First column of each line is supposed to have the field name, and is ignored, too. The passed character is the expected field sepparator (usually, comma or tab)

Parameters

in	*fname*	The name of the csv file to read
in	*vtype*	The data type to be stored
in	*csep*	The character used as field sepparator

4.3.2.6 SparseMatrix() [6/6]

```
template<typename T >
SparseMatrix< T >::SparseMatrix (
            const SparseMatrix< T > & other )
```

Copy constructor

Parameters

in	*other*	Reference to the SparseMatrix to be copied

4.3.2.7 ~SparseMatrix()

```
template<typename T >
SparseMatrix< T >::~SparseMatrix
```

Destructor

4.3.3 Member Function Documentation

4.3.3.1 Get()

```
template<typename T >
T SparseMatrix< T >::Get (
            indextype r,
            indextype c ) const
```

Function to get acess to an element

Parameters

in	*r*	The row to access
in	*c*	The columns to access

Returns

value at (r,c) of matrix of type T

4.3.3.2 GetMarksOfSparseRow()

```
template<typename T >
void SparseMatrix< T >::GetMarksOfSparseRow (
            indextype r,
            unsigned char * m,
            unsigned char s )
```

Function to get from a sparse row a pointer to an array of marks signalling where the non-zero elements are. The array of marks will be changed OR'ing the passed mark to each place where there is a value. The pointer to the marks is not returned since that way it do not need to be booked. It is passed as a parameter and is supposed to be properly allocated.

Parameters

in	*r*	The row to get
out	$*m$	Pointer to the marks
in	*s*	The value to be OR'ed to each place at the mark array

4.3.3.3 GetRow()

```
template<typename T >
void SparseMatrix< T >::GetRow (
            indextype r,
            T * v )
```

Function to get a row as a pointer to the content type. Row is not a sparse but a full vector with zeros when needed. The pointer is not returned since that way it does not need to be booked. The pointer to hold result is passed as parameter and it is supposed to be properly allocated.

Parameters

in	*r*	The row to get
out	**v*	Pointer to the result of type *T

4.3.3.4 GetSparseRow()

```
template<typename T >
void SparseMatrix< T >::GetSparseRow (
            indextype r,
            unsigned char * m,
            unsigned char s,
            T * v )
```

Function to get a sparse row as a pointer to the content type plus a pointer to an array of marks.
The content will have the values. The array of marks will be changed OR'ing the passed mark to each place where there is a value. The pointers to the values and marks are not returned since that way they do not need to be booked. They are passed as parameters and both are supposed to be properly allocated.
This strange way of storing data has been choosen because it will be specially suitable to calculate distance between sparse vectors

Parameters

in	*r*	The row to get
out	**v*	Pointer to the values
out	**m*	Pointer to the marks
in	*s*	The value to be OR'ed to each place at the mark array

4.3.3.5 GetUsedMemoryMB()

```
template<typename T >
float SparseMatrix< T >::GetUsedMemoryMB
```

Function to get memory in MB used by this sparse matrix (including values and additional indexes)

Returns

The amount of memory in MB

4.3.3.6 operator"!=()

```
template<typename T >
SparseMatrix< T > & SparseMatrix< T >::operator!= (
            const SparseMatrix< T > & other )
```

Transpose-assignment

Parameters

in	*other*	Reference to the SparseMatrix to be assigned

Returns

Reference to the newly created SparseMatrix, which is the transpose of the passed one

4.3.3.7 operator=()

```
template<typename T >
SparseMatrix< T > & SparseMatrix< T >::operator= (
            const SparseMatrix< T > & other )
```

Assignment operator

Parameters

in	*other*	Reference to the SparseMatrix to be assigned

Returns

Reference to the newly created SparseMatrix

4.3.3.8 Resize()

```
template<typename T >
void SparseMatrix< T >::Resize (
            indextype newnr,
            indextype newnc )
```

Function to resize the matrix
WARNING: previous content, if any, IS LOST (to be reviewed)

Parameters

in	*newnr*	New number of rows
in	*newnc*	New number of cols

4.3.3.9 SelfColNorm()

```
template<typename T >
void SparseMatrix< T >::SelfColNorm (
            std::string ctype )
```

Function to alter the internal values of the matrix so that each column is normalized according to the requested normalization type The purpose of this function can be achieved with a loop using Set and Get, but using the internal structure makes the task much faster
Normally, this function will not be used outside the context of bioinformatics where these normalizations are standard

Parameters

in	*ctype*	The requested type of normalization: rawn, log1 or log1n

4.3.3.10 SelfRowNorm()

```
template<typename T >
void SparseMatrix< T >::SelfRowNorm (
            std::string ctype )
```

Function to alter the internal values of the matrix so that each row is normalized according to the requested normalization type The purpose of this function can be achieved with a loop using Set and Get, but using the internal structure makes the task much faster
Normally, this function will not be used outside the context of bioinformatics where these normalizations are standard

Parameters

in	*ctype*	The requested type of normalization: rawn, log1 or log1n

4.3.3.11 Set()

```
template<typename T >
void SparseMatrix< T >::Set (
            indextype r,
            indextype c,
            T v )
```

Function to set an element

Parameters

in	*r*	The row of the element to be set
in	*c*	The column of the element to be set
in	*v*	The value to be set, of type T

4.3.3.12 SetRow()

```
template<typename T >
void SparseMatrix< T >::SetRow (
            indextype r,
            std::vector< indextype > vc,
            std::vector< T > v )
```

Function to set a row (as two vectors of locations and values)

Parameters

in	*r*	The row to be set
in	*vc*	The vector with the columns to be set
in	*v*	The vector with the corresponding values to be set. Must be the same length as vc

4.3.3.13 WriteBin()

```
template<typename T >
void SparseMatrix< T >::WriteBin (
            std::string fname )
```

Function to write the matrix content into a binary file
For the header format, see the documentation of JMatrix

After the header comes the content as raw data, by rows, with this content for each row:

- indextype ncr: number of non-zero entries of this row
- ncr values of indextype with the numbers of the columns of this row occupied by non-zero entries
- ncr elements of the current value type (the values of all non-zero entries of this row).

Parameters

in	*fname*	The name of the file to write

4.3.3.14 WriteCsv()

```
template<typename T >
void SparseMatrix< T >::WriteCsv (
            std::string fname,
            char csep = ',',
            bool withquotes = false )
```

Function to write the matrix content to a CSV file

Parameters

in	*fname*	The name of the file to write
in	*csep*	The separator character between fields (default: , (comma))
in	*withquotes*	Boolean value to indicate if field names in .csv must be written surrounded by quotes.

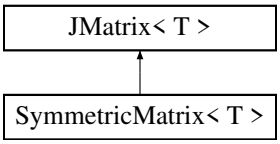
The documentation for this class was generated from the following files:

- src/headers/sparsematrix.h
- src/library/sparsematrix.cpp

4.4 SymmetricMatrix< T > Class Template Reference

```
#include <symmetricmatrix.h>
```

Inheritance diagram for SymmetricMatrix< T >:



Public Member Functions

- SymmetricMatrix ()
- SymmetricMatrix (indextype nrows)
- SymmetricMatrix (indextype nrows, bool warn)
- SymmetricMatrix (std::string fname)
- SymmetricMatrix (std::string fname, bool warn)
- void Resize (indextype newnr)
- SymmetricMatrix (const SymmetricMatrix< T > &other)
- ~SymmetricMatrix ()
- SymmetricMatrix< T > & operator= (const SymmetricMatrix< T > &other)
- bool TestDistDisMat ()
- T Get (indextype r, indextype c)
- void Set (indextype r, indextype c, T v)
- T GetRowSum (indextype r)
- void WriteCsv (std::string fname, char csep=',', bool withquotes=false)
- void WriteBin (std::string fname)
- float GetUsedMemoryMB ()

Additional Inherited Members

4.4.1 Detailed Description

template<typename T>
class SymmetricMatrix< T >

@SymmetricMatrix Class to hold arbitrarily big symmetric square matrices. For a matrix of size NxN, only Nx(N+1)/2 elements are stored.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 SymmetricMatrix() [1/6]

```
template<typename T >
SymmetricMatrix< T >::SymmetricMatrix
```

Default constructor

4.4.2.2 SymmetricMatrix() [2/6]

```
template<typename T >
SymmetricMatrix< T >::SymmetricMatrix (
            indextype nrows )
```

Constructor with number of rows/columns (the same, it is square)

Parameters

in	*nrows*	Number of rows

4.4.2.3 SymmetricMatrix() [3/6]

```
template<typename T >
SymmetricMatrix< T >::SymmetricMatrix (
            indextype nrows,
            bool warn )
```

Constructor with number of rows/columns (the same, it is square) giving memory warnings

Parameters

in	*nrows*	Number of rows

Parameters

in	*warn*	Boolean value to give memory warnings The idea is to use this form of constructor with warn=true, which internally calls MemoryWarnings, if one suspects the matrix to be constructed might provoke memory problems.

4.4.2.4 SymmetricMatrix() [4/6]

```
template<typename T >
SymmetricMatrix< T >::SymmetricMatrix (
            std::string fname )
```

Constructor to fill the matrix content from a binary file
Binary file header as explained in the documetation to WriteBin

PRELIMINARY VERSION. ASSUMES SAME ENDIANESS FOR WRITER AND READER MACHINE

Parameters

in	*fname*	The name of the file to read

4.4.2.5 SymmetricMatrix() [5/6]

```
template<typename T >
SymmetricMatrix< T >::SymmetricMatrix (
            std::string fname,
            bool warn )
```

Constructor to fill the matrix content from a binary file with warnings
Binary file header as explained in the documetation to WriteBin

PRELIMINARY VERSION. ASSUMES SAME ENDIANESS FOR WRITER AND READER MACHINE

Parameters

in	*fname*	The name of the file to read
in	*warn*	Boolean value to give memory warnings

4.4.2.6 SymmetricMatrix() [6/6]

```
template<typename T >
SymmetricMatrix< T >::SymmetricMatrix (
            const SymmetricMatrix< T > & other )
```

Copy constructor

Parameters

in	*other*	Reference to the SymmetricMatrix to be copied

4.4.2.7 ~SymmetricMatrix()

```
template<typename T >
SymmetricMatrix< T >::~SymmetricMatrix
```

Destructor

4.4.3 Member Function Documentation

4.4.3.1 Get()

```
template<typename T >
T SymmetricMatrix< T >::Get (
            indextype r,
            indextype c )
```

Function to get acess to an element

Parameters

in	*r*	The row to access
in	*c*	The columns to access

Returns

value at (r,c) of matrix, of type T

4.4.3.2 GetRowSum()

```
template<typename T >
T SymmetricMatrix< T >::GetRowSum (
            indextype r )
```

Function to get the sum of a row (used frequently by PAM)

Parameters

in	*r*	The row whose sum we want

Returns

The sum of all columns of row r, of type T

4.4.3.3 GetUsedMemoryMB()

```
template<typename T >
float SymmetricMatrix< T >::GetUsedMemoryMB
```

Function to get memory in MB used by this symmetric matrix

Returns

The amount of memory in MB

4.4.3.4 operator=()

```
template<typename T >
SymmetricMatrix< T > & SymmetricMatrix< T >::operator= (
            const SymmetricMatrix< T > & other )
```

Assignment operator

Parameters

in	*other*	Reference to the SymmetricMatrix to be assigned

Returns

Reference to the newly created SymmetricMatrix

4.4.3.5 Resize()

```
template<typename T >
void SymmetricMatrix< T >::Resize (
            indextype newnr )
```

Function to resize the matrix
WARNING: previous content, if any, IS LOST (to be reviewed)

Parameters

in	*newnr*	New number of rows (and columns)

4.4.3.6 Set()

```
template<typename T >
void SymmetricMatrix< T >::Set (
            indextype r,
            indextype c,
            T v )
```

Function to set an element

Parameters

in	*r*	The row to access
in	*c*	The columns to access
in	*T*	The value to be set, of type T

4.4.3.7 TestDistDisMat()

```
template<typename T >
bool SymmetricMatrix< T >::TestDistDisMat
```

Test of correctness
This is meant to test if the symmetric matrix is a distance or dissimilarity matrix. It checks that all elements in the main diagonal are 0 and all outside the main diagonal are stricty possitive

Returns

true if the matrix can be a distance or dissimilarity matrix. false otherwise.

4.4.3.8 WriteBin()

```
template<typename T >
void SymmetricMatrix< T >::WriteBin (
            std::string fname )
```

Function to write the matrix content to a binary file
See format at documentation of JMatrix::WriteBin

Parameters

in	*fname*	The name of the file to write

4.4.3.9 WriteCsv()

```
template<typename T >
void SymmetricMatrix< T >::WriteCsv (
            std::string fname,
            char csep = ',',
            bool withquotes = false )
```

Function to write the matrix content to a CSV file

Parameters

in	*fname*	The name of the file to write
in	*csep*	The separator character between fields (default: , (comma))
in	*withquotes*	Boolean value to indicate if field names in .csv must be written surrounded by quotes.

The documentation for this class was generated from the following files:

- src/headers/symmetricmatrix.h
- src/library/symmetricmatrix.cpp

Chapter 5

File Documentation

5.1 src/examples/jmat.cpp File Reference

```
#include "../headers/debugpar.h"
#include "../headers/jmatrix.h"
#include "../headers/apitocommands.h"
```

Functions

- int main (int argc, char *argv[ ])

5.1.1 Detailed Description

jmat

See program use in the documention to main() below

NOTE: The includes in this source file are for compilation of this program as an example together with the library,
before the library itself is installed. Once you have installed the library (assuming headers are in
/usr/local/include, lib is in /usr/local/lib or in other place included in your compiler search path)
you should substitute this by

#include <jmatrixlib/debugpar.h> etc...

and compile with something like

g++ -Wall jmat.cpp -o jmat -ljmatrix

5.1.2 Function Documentation

5.1.2.1 main()

```
int main (
            int argc,
            char * argv[] )
```

jmat

A program to manipulate matrices created with the jmatrix library and written to the disk as binary files.

The program must be called as

```
jmat command matrix_file other_options -o out_matrix file
```

where command is one of a predefined list (see below) which is followed by the matrix to be manipulated, other relevant options for the particular command and (optionally) the -o option with the result of the command.
If -o option is not given, the result is dumped to the console in ASCII
other_options are options dependent on the command (call 'jmat any_command' for specific information)
Also, remember that if this program is called as **jmatd** (symbolic link to jmat) you will get debugging messages in the console.

Possible commands are:

```
jmat info matrix_file [-o info_file.txt]
```

Dumps in ASCII to console or to info_file.txt information about the jmatrix in file matrix_file.

```
jmat rownum matrix_file n -o res_file
```

Creates a jmatrix file with the n-th row of the jmatrix in file matrix_file (index n from 0).

```
jmat rownums matrix_file n1,n2,... -o res_file
```

Creates a jmatrix file with the rows in the comma-sepparated list of the jmatrix in file matrix_file (indices n1,n2,... from 0).

```
jmat rowname matrix_file rname -o res_file
```

Creates a jmatrix file with the row named 'rname' of the jmatrix in file matrix_file (index n from 0).

```
jmat rownames matrix_file rn1,rn2,... -o res_file
```

Creates a jmatrix file, with the rows named 'rn1','rn2'... in the comma-sepparated list of the jmatrix n file matrix_file.

```
jmat colnum matrix_file n -o res_file
```

Creates a jmatrix file, with the n-th column of the jmatrix in file matrix_file (index n from 0).

```
jmat colnums matrix_file n1,n2,... -o res_file
```

Creates a jmatrix file, with the columns in the comma-sepparated list of the jmatrix in file matrix_file (indices form 0).

```
jmat colname matrix_file cname -o res_file
```

Creates a jmatrix file, with the column named 'cname' of the jmatrix in file matrix_file

```
jmat colnames matrix_file cn1,cn2,... -o res_file
```

Creates a jmatrix file, with the columns named 'cn1','cn2'... in the comma-sepparated list of the jmatrix in file matrix_file.

```
jmat subdiag symmetric_matrix_file -o res_file
```

Creates a jmatrix file with the lower-triangular part (not including the diagonal) of the symmetric matrix in the input.

```
jmat getrnames matrix_file -o res_file.txt
```

Dumps to ASCII file res_file.txt the names of the rows in the input matrix.

```
jmat getcnames matrix_file -o res_file.txt
```

Dumps to ASCII file res_file.txt the names of the columns in the input matrix.

```
jmat setrnames matrix_file rownames_file.txt -o res_file
```

Creates a copy of the original jmatrix setting or changing the row names to those given in file rownames_file.txt, which must have as many lines as rows in input matrix.

```
jmat setcnames matrix_file colnames_file.txt -o res_file
```

Creates a copy of the original jmatrix setting or changing the column names to those given in file colnames_file.txt, which must have as many lines as columns in input matrix.

```
jmat setrcnames matrix_file rownames_file.txt colnames_file.txt -o res_file
```

Creates a copy of the original jmatrix setting or changing the row and column names to those given in files rownames_file.txt and colnames_file.txt respectively, which must have as many lines as rows/columns in input matrix.

```
jmat csvdump matrix_file format -o res_file.csv
```

Dumps the content of the matrix in the input file to the .csv output file. Format must be one of these strings:
cn Comma as separator, row/column names not surrounded by double quotes.
cq Comma as separator, row/column names surrounded by double quotes.
tn Tab as separator, row/column names not surrounded by double quotes.
tq Tab as separator, row/column names surrounded by double quotes.

```
jmat csvread input_file.csv sepchar mtype valtype -o res_file
```

Reads the input file, which must be a csv file, and creates a binary jmatrix file with its content.
sepchar must be c or t to indicate that the expected field separator will be a comma or a tab, respectively.
mtype must be one of the strings 'full' or 'sparse'. Symmetric matrices cannot be read as such from a CSV file.
valtype must be one of the strings 'u8','s8','u16','s16','u32','s32','u64','s64','f','d' or 'ld'.
These stands for unsigned/signed integers of 8,16,32 or 64 bits, float, double, or long double datatypes, respectively.

5.2 src/headers/apitocommands.h File Reference

```
#include <string>
#include <vector>
#include "indextype.h"
```

Functions

- void JMatInfo (std::string iname, std::string oname)
- void JGetNumRow (std::string iname, std::string oname, indextype numrow)
- void JGetNumCol (std::string iname, std::string oname, indextype numcol)
- void JGetNameRow (std::string iname, std::string oname, std::string namerow)
- void JGetNameCol (std::string iname, std::string oname, std::string namecol)
- void JGetNumsRow (std::string iname, std::string oname, std::vector< indextype > lrows)
- void JGetNumsCol (std::string iname, std::string oname, std::vector< indextype > lcols)
- void JGetNamesRow (std::string iname, std::string oname, std::vector< std::string > lrows)
- void JGetNamesCol (std::string iname, std::string oname, std::vector< std::string > lcols)
- void JGetSubDiag (std::string iname, std::string oname)
- void JGetRowNames (std::string iname, std::string oname)
- void JGetColNames (std::string iname, std::string oname)
- void JSetRowNames (std::string iname, std::string oname, std::vector< std::string > rnames)
- void JSetColNames (std::string iname, std::string oname, std::vector< std::string > cnames)
- void JSetRowColNames (std::string iname, std::string oname, std::vector< std::string > rnames, std::vector< std::string > cnames)
- void JCsvDump (std::string iname, std::string oname, char sep, bool with_quotes)
- void JCsvToJMat (std::string iname, std::string oname, char sep, unsigned char mtype, unsigned char ctype)

5.2.1 Function Documentation

5.2.1.1 JCsvDump()

```
void JCsvDump (
            std::string iname,
            std::string oname,
            char sep,
            bool with_quotes )
```

Function to dump the JMatrix contained in a binary file to as ASCII csv file

Parameters

in	*iname*	Name of the JMatrix binary file with the matrix
in	*oname*	Name of the csv file to generat
in	*char*	The character to be used as sepparator
in	*withquotes*	Boolean value to indicate if field names in .csv must be written surrounded by quotes.

5.2.1.2 JCsvToJMat()

```
void JCsvToJMat (
            std::string iname,
            std::string oname,
            char sep,
            unsigned char mtype,
            unsigned char ctype )
```

Function to generate a binary JMatrix file from an ASCII csv file
First line of csv is supposed to contain the column names, starting with an empty field
First field of each line from the second line and on is supposed to contain the name of such row

Parameters

in	*iname*	CSV file with the data
in	*oname*	Name of the binary file to contain the created JMatrix
in	*sep*	The character that csv file uses as field sepparator
in	*mtype*	The type of the JMatrix. Possible values: 'full', 'sparse' or 'symmetric'
in	*ctype*	The data type to store the read values (value type of the JMatrix). Possible values: 'u8','s8','u16','s16','u32','s32','u64','s64','f','d' or 'ld'

5.2.1.3 JGetColNames()

```
void JGetColNames (
            std::string iname,
            std::string oname )
```

Function to write to an ASCII text file the names of the columns of the JMatrix stored in a binary file
The matrix is not loaded into memory

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the ASCII file to contain the column names, one at each line

5.2.1.4 JGetNameCol()

```
void JGetNameCol (
            std::string iname,
            std::string oname,
            std::string namecol )
```

Function to get a column by name and write it as a JMatrix in a binary file, assuming the input matrix has column names and such name exists

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the binary file to write the row contents
in	*namecol*	The name of the column we want to extract

5.2.1.5 JGetNameRow()

```
void JGetNameRow (
            std::string iname,
            std::string oname,
            std::string namerow )
```

Function to get a row by name and write it as a JMatrix in a binary file, assuming the input matrix has row names and such name exists

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the binary file to write the row contents
in	*namerow*	The name of the row we want to extract

5.2.1.6 JGetNamesCol()

```
void JGetNamesCol (
            std::string iname,
            std::string oname,
            std::vector< std::string > lcols )
```

Function to get several columns by their names and write them as a JMatrix in a binary file

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the binary file to write the rows as a matrix
in	*lcols*	A vector of strings with the names of the columns we want to extract

5.2.1.7 JGetNamesRow()

```
void JGetNamesRow (
            std::string iname,
```

```
            std::string oname,
            std::vector< std::string > lrows )
```

Function to get several rows by their names and write them as a JMatrix in a binary file

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the binary file to write the rows as a matrix
in	*lrows*	A vector of strings with the names of the rows we want to extract

5.2.1.8 JGetNumCol()

```
void JGetNumCol (
            std::string iname,
            std::string oname,
            indextype numcol )
```

Function to get a column by number and write it as a JMatrix in a binary file

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the binary file to write the column contents
in	*numrow*	The number of the column (0-based index) we want to extract

5.2.1.9 JGetNumRow()

```
void JGetNumRow (
            std::string iname,
            std::string oname,
            indextype numrow )
```

Function to get a row by number and write it as a JMatrix in a binary file

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the binary file to write the row contents
in	*numrow*	The number of the row (0-based index) we want to extract

5.2.1.10 JGetNumsCol()

```
void JGetNumsCol (
            std::string iname,
            std::string oname,
            std::vector< indextype > lcols )
```

Function to get several columns by number and write them as a JMatrix in a binary file

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the binary file to write the columns as a matrix
in	*lcols*	A vector with the numbers of the columns (0-based index) we want to extract

5.2.1.11 JGetNumsRow()

```
void JGetNumsRow (
            std::string iname,
            std::string oname,
            std::vector< indextype > lrows )
```

Function to get several rows by number and write them as a JMatrix in a binary file

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the binary file to write the rows as a matrix
in	*lrows*	A vector with the numbers of the rows (0-based index) we want to extract

5.2.1.12 JGetRowNames()

```
void JGetRowNames (
            std::string iname,
            std::string oname )
```

Function to write to an ASCII text file the names of the rows of the JMatrix stored in a binary file
The matrix is not loaded into memory

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the ASCII file to contain the row names, one at each line

5.2.1.13 JGetSubDiag()

```
void JGetSubDiag (
            std::string iname,
            std::string oname )
```

Function to get the subdiagonal of a SymmetricMatrix of size (n x n) stored in a binary file and write them as a vector of one row and n x (n-1)/2 columns (rows under the main diagonal, without the diagonal itself) stored in row-major order

Parameters

in	*iname*	Name of the SymmatricMatrix binary file
in	*oname*	Name of the binary file to write the subdiagonal elements as a 1-row matrix

5.2.1.14 JMatInfo()

```
void JMatInfo (
            std::string iname,
            std::string oname )
```

Function to get information about the JMatrix stored in a binary file and store such information in a text file

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the text file to write the information

5.2.1.15 JSetColNames()

```
void JSetColNames (
            std::string iname,
            std::string oname,
            std::vector< std::string > cnames )
```

Function to create a copy of the original jmatrix setting or changing the column names to those given in a vector of strings, which must have as many elements as columns in the input matrix

Parameters

in	*iname*	Name of the JMatrix binary file with the original matrix
in	*oname*	Name of the JMatrix binary file with the copy matrix with new column names
in	*cnames*	Vector of strings with the new column names

5.2.1.16 JSetRowColNames()

```
void JSetRowColNames (
            std::string iname,
            std::string oname,
            std::vector< std::string > rnames,
            std::vector< std::string > cnames )
```

Function to create a copy of the original jmatrix setting or changing the column and row names to those given in vectors of strings, which must have as many elements as rows and columns respectively in the input matrix

Parameters

in	*iname*	Name of the JMatrix binary file with the original matrix
in	*oname*	Name of the JMatrix binary file with the copy matrix with new column names
in	*rnames*	Vector of strings with the new row names
in	*cnames*	Vector of strings with the new column names

5.2.1.17 JSetRowNames()

```
void JSetRowNames (
            std::string iname,
            std::string oname,
            std::vector< std::string > rnames )
```

Function to create a copy of the original jmatrix setting or changing the row names to those given in a vector of strings, which must have as many elements as rows in the input matrix

Parameters

in	*iname*	Name of the JMatrix binary file with the original matrix
in	*oname*	Name of the JMatrix binary file with the copy matrix with new row names
in	*rnames*	Vector of strings with the new row names

5.3 src/headers/debugpar.h File Reference

```
#include <iostream>
#include <string>
```

Functions

- void JMatrixSetDebug (bool deb)
- void JMatrixStop (std::string errortext)
- void JMatrixWarning (std::string warntext)

Variables

- const unsigned char NODEBUG =0x0
- const unsigned char DEBJM =0x01

5.3.1 Function Documentation

5.3.1.1 JMatrixSetDebug()

```
void JMatrixSetDebug (
            bool deb )
```

Sets the debug state to get messages or not

Parameters

in	*deb*	true to get messages, false to suppress them. Default state is false.

5.3.1.2 JMatrixStop()

```
void JMatrixStop (
            std::string errortext )
```

Sends an error message to the console and stops the program that is using the library

Parameters

in	*errortext*	The text of the message to be shown. It will appear after a standard message saying that is comes from this library.

5.3.1.3 JMatrixWarning()

```
void JMatrixWarning (
            std::string warntext )
```

Sends a warning message to the console and goes on with the program that is using the library

Parameters

in	*errortext*	The text of the message to be shown. It will appear after a standard message saying that is comes from this library.

5.3.2 Variable Documentation

5.3.2.1 DEBJM

```
const unsigned char DEBJM =0x01
```

These are constants to allow selective debug by library. Each library will print messages or not using a test with logical AND between its particular constant and the DEB global variable. This allows the use of the system either in each separate package or in the global one

5.3.2.2 NODEBUG

```
const unsigned char NODEBUG =0x0
```

These are constants to allow selective debug by library. Each library will print messages or not using a test with logical AND between its particular constant and the DEB global variable. This allows the use of the system either in each separate package or in the global one

5.4 src/headers/fullmatrix.h File Reference

```
#include "jmatrix.h"
#include "memhelper.h"
```

Classes

- class FullMatrix< T >

5.5 src/headers/indextype.h File Reference

Typedefs

- typedef unsigned int indextype

5.5.1 Typedef Documentation

5.5.1.1 indextype

```
unsigned int indextype
```

This is the type of the indexes. It is left fixed, but using a typedef allows easy recompilation of the library if we decide otherwise in the future.
It is quite unlikely we need to change indextype, unless, to save some memory and if you know for sure that there will be no more than 65536 individuals, can be defined as unsigned short. But this might provoke hidden problems in other places so, please, think twice before...
Increasing it to 128 bits (unsigned long long) would be possible, but if you need such size of matrices, better don't use this library...

5.6 src/headers/jmatrix.h File Reference

```
#include <iostream>
#include <string>
#include <cstring>
#include <cmath>
#include <sstream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <type_traits>
#include <sys/stat.h>
#include "debugpar.h"
#include "indextype.h"
#include "matinfo.h"
```

Classes

- class JMatrix< T >

Macros

- #define **WITH_CHECKS_MATRIXSP** 1

Functions

- unsigned char ThisMachineEndianness ()
- unsigned long long GetFileSize (std::string fname)
- void PositionsInFile (std::string fname, unsigned long long *start_of_metadata, unsigned long long *start_↩ of_comment)
- std::string MatrixTypeName (unsigned char matrixtypeident)

  Auxiliary functions to be used for error printing.
- std::string DataTypeName (unsigned char datatypeident)
- std::string MetadataInfo (unsigned char metadatainfo)
- int SizeOfType (unsigned char datatypeident)
- std::string **FixQuotes** (std::string s, bool withquotes)

Variables

- const unsigned short HEADER_SIZE =128

- const unsigned char MTYPENOTYPE =0x0F
- const unsigned char MTYPEFULL =0x00
- const unsigned char MTYPESPARSE =0x01
- const unsigned char MTYPESYMMETRIC =0x02

- const unsigned char NOTYPE =0x0F
- const unsigned char UCTYPE =0x00
- const unsigned char SCTYPE =0x01
- const unsigned char USTYPE =0x02
- const unsigned char SSTYPE =0x03
- const unsigned char UITYPE =0x04
- const unsigned char SITYPE =0x05
- const unsigned char ULTYPE =0x06
- const unsigned char SLTYPE =0x07
- const unsigned char ULLTYPE =0x08
- const unsigned char SLLTYPE =0x09
- const unsigned char FTYPE =0x0A
- const unsigned char DTYPE =0x0B
- const unsigned char LDTYPE =0x0C

- const unsigned char BIGEND =0x00
- const unsigned char LITEND =0xF0

- const int READ_OK =0
- const int ERROR_READING_STRINGS =1
- const int ERROR_READING_ROW_NAMES =2
- const int ERROR_READING_COL_NAMES =3
- const int ERROR_READING_SEP_MARK =4
- const unsigned int MAX_LEN_NAME =1023
- const unsigned int COMMENT_SIZE =1024
- const unsigned char NO_METADATA =0x00
- const unsigned char ROW_NAMES =0x01
- const unsigned char COL_NAMES =0x02
- const unsigned char COMMENT =0x04
- const unsigned int BLOCKSEP_LEN =4
- const unsigned char BLOCK_MARK =0xFF
- const unsigned char BLOCKSEP [BLOCKSEP_LEN] ={BLOCK_MARK,0x45,0x42,BLOCK_MARK}

5.6.1 Function Documentation

5.6.1.1 DataTypeName()

```
std::string DataTypeName (
            unsigned char datatypeident )
```

Returns the name of the data type whose type (as indentifier) is passed

Parameters

datatypeident.	The data type of the data in the matrix (unsigned char, char,...) as defined by the former constants.

Returns

A human-meaningful string decribing the data type

5.6.1.2 GetFileSize()

```
unsigned long long GetFileSize (
            std::string fname )
```

Returns the file size of a file in a sufficiently large number (unsigned long long) in a way (hopefully) independent of the operating system and of the architecture

Parameters

File	path

Returns

File size

5.6.1.3 MatrixTypeName()

```
std::string MatrixTypeName (
            unsigned char matrixtypeident )
```

Auxiliary functions to be used for error printing.

Returns the name of the matrix whose type (as indentifier) is passed

Parameters

matrixtypeident.	The type of the matrix (full, sparse, symmetric) as defined by the former constants.

Returns

A human-meaningful string decribing the type

5.6.1.4 MetadataInfo()

```
std::string MetadataInfo (
            unsigned char metadatainfo )
```

Returns a message to interpret the presence of metadata

Parameters

metadatainfo.	The constant for information on which metadata are present as defined by the former constants.

Returns

A human-meaningful string decribing the present metadata

5.6.1.5 PositionsInFile()

```
void PositionsInFile (
            std::string fname,
            unsigned long long * start_of_metadata,
            unsigned long long * start_of_comment )
```

Returns the positions of the start of metadata and start of comments (included inside metadata) as absolute positions measured in bytes from the beginning of the file

Parameters

File	path
∗*start_of_metadata*	
∗*start_of_comment*	

5.6.1.6 SizeOfType()

```
int SizeOfType (
            unsigned char datatypeident )
```

Returns the size in bytes of the data type whose type (as indentifier) is passed

Parameters

datatypeident.	The data type of the data in the matrix (unsigned char, char,...) as defined by the former constants.

Returns

The size in bytes of one element of the passed data type.

5.6.1.7 ThisMachineEndianness()

```
unsigned char ThisMachineEndianness ( )
```

Returns the endianness of the machine where this function is called

Returns

Either the constant BIGEND or the constant LITEND

5.6.2 Variable Documentation

5.6.2.1 BIGEND

```
const unsigned char BIGEND =0x00
```

Constants for the possible endianness of a machine Only two values allowed for big and little endian Big endian

5.6.2.2 BLOCK_MARK

```
const unsigned char BLOCK_MARK =0xFF
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.3 BLOCKSEP

```
const unsigned char BLOCKSEP[BLOCKSEP_LEN] ={BLOCK_MARK,0x45,0x42,BLOCK_MARK}
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.4 BLOCKSEP_LEN

```
const unsigned int BLOCKSEP_LEN =4
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.5 COL_NAMES

```
const unsigned char COL_NAMES =0x02
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.6 COMMENT

```
const unsigned char COMMENT =0x04
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.7 COMMENT_SIZE

```
const unsigned int COMMENT_SIZE =1024
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.8 DTYPE

```
const unsigned char DTYPE =0x0B
```

double

5.6.2.9 ERROR_READING_COL_NAMES

```
const int ERROR_READING_COL_NAMES =3
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.10 ERROR_READING_ROW_NAMES

```
const int ERROR_READING_ROW_NAMES =2
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.11 ERROR_READING_SEP_MARK

```
const int ERROR_READING_SEP_MARK =4
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.12 ERROR_READING_STRINGS

```
const int ERROR_READING_STRINGS =1
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.13 FTYPE

```
const unsigned char FTYPE =0x0A
```

float

5.6.2.14 HEADER_SIZE

```
const unsigned short HEADER_SIZE =128
```

The header size. We fix a header of 128 bytes. We don't need so much, but just in case in the future...

5.6.2.15 LDTYPE

```
const unsigned char LDTYPE =0x0C
```

long double

5.6.2.16 LITEND

```
const unsigned char LITEND =0xF0
```

Little endian

5.6.2.17 MAX_LEN_NAME

```
const unsigned int MAX_LEN_NAME =1023
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.18 MTYPEFULL

```
const unsigned char MTYPEFULL =0x00
```

Full matrix

5.6.2.19 MTYPENOTYPE

```
const unsigned char MTYPENOTYPE =0x0F
```

Constants for the possible matrix types Currently, they are no type (for errors), full matrix, sparse matrix and symmetric matrix. No matrix type

5.6.2.20 MTYPESPARSE

```
const unsigned char MTYPESPARSE =0x01
```

Sparse matrix

5.6.2.21 MTYPESYMMETRIC

```
const unsigned char MTYPESYMMETRIC =0x02
```

Symmetric matrix

5.6.2.22 NO_METADATA

```
const unsigned char NO_METADATA =0x00
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.23 NOTYPE

```
const unsigned char NOTYPE =0x0F
```

Constants for the possible data types a matrix can hold. These are (apart of the no type for errors) integer types: char (8 bits), short int (16 bits), int (32 bits), long (32 bits), long long (64 bits) with their signed versions and float types in IEEE-754 format: float (32 bits), double (64 bits) and long double (128 bits) No data type

5.6.2.24 READ_OK

```
const int READ_OK =0
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.25 ROW_NAMES

```
const unsigned char ROW_NAMES =0x01
```

Constants for information about the metadata included with the matrix Possible metadata stored are currently names of rows and names of columns Errors are returned with these constants in case of bad reads

5.6.2.26 SCTYPE

```
const unsigned char SCTYPE =0x01
```

char

5.6.2.27 SITYPE

```
const unsigned char SITYPE =0x05
```

int

5.6.2.28 SLLTYPE

```
const unsigned char SLLTYPE =0x09
```

long long

5.6.2.29 SLTYPE

```
const unsigned char SLTYPE =0x07
```

long

5.6.2.30 SSTYPE

```
const unsigned char SSTYPE =0x03
```

short

5.6.2.31 UCTYPE

```
const unsigned char UCTYPE =0x00
```

unsigned char

5.6.2.32 UITYPE

```
const unsigned char UITYPE =0x04
```

unsigned int

5.6.2.33 ULLTYPE

```
const unsigned char ULLTYPE =0x08
```

unsigned long long

5.6.2.34 ULTYPE

```
const unsigned char ULTYPE =0x06
```

unsigned long

5.6.2.35 USTYPE

```
const unsigned char USTYPE =0x02
```

unsigned short

5.7 src/headers/matinfo.h File Reference

```
#include <iostream>
#include <string>
#include "indextype.h"
```

Functions

- void JMatInfo (std::string fname, std::string fres="")

- void MatrixType (std::string fname, unsigned char &mtype)
- void MatrixType (std::string fname, unsigned char &mtype, unsigned char &ctype)
- void MatrixType (std::string fname, unsigned char &mtype, unsigned char &ctype, unsigned char &endianness)
- void MatrixType (std::string fname, unsigned char &mtype, unsigned char &ctype, unsigned char &endianness, unsigned char &mdinf)
- void MatrixType (std::string fname, unsigned char &mtype, unsigned char &ctype, unsigned char &endianness, unsigned char &mdinf, indextype &nrows, indextype &ncols)

5.7.1 Function Documentation

5.7.1.1 JMatInfo()

```
void JMatInfo (
            std::string fname,
            std::string fres = "" )
```

Gives information about the JMatrix stored in a file

Parameters

in	*fname*	The name of the binary file that contains the matrix
in	*fres*	The name of the text file that will contain the information, or the empty string to show the information in the console

Function to get information about the JMatrix stored in a binary file and store such information in a text file

Parameters

in	*iname*	Name of the JMatrix binary file
in	*oname*	Name of the text file to write the information

5.7.1.2 MatrixType() [1/5]

```
void MatrixType (
            std::string fname,
            unsigned char & mtype )
```

Auxiliary funcions to check characteristics of matrix stored in binary file looking only at its header. Matrix in file is NOT loaded into memory.

Parameters

in	*fname*	Name of the binary file
out	*mtype*	Returns matrix type (full,sparse,symmetric)
out	*ctype*	Returns matrix data type
out	*endianness*	Returns endianness (big,little) of the data stored in the matrix
out	*nrows*	Returns the number of rows
out	*ncols*	Returns the number of columns
out	*mdinf*	Returns the signal for the presence of metadata

5.7.1.3 MatrixType() [2/5]

```
void MatrixType (
            std::string fname,
            unsigned char & mtype,
            unsigned char & ctype )
```

Auxiliary funcions to check characteristics of matrix stored in binary file looking only at its header. Matrix in file is NOT loaded into memory.

Parameters

in	*fname*	Name of the binary file

Parameters

out	*mtype*	Returns matrix type (full,sparse,symmetric)
out	*ctype*	Returns matrix data type
out	*endianness*	Returns endianness (big,little) of the data stored in the matrix
out	*nrows*	Returns the number of rows
out	*ncols*	Returns the number of columns
out	*mdinf*	Returns the signal for the presence of metadata

5.7.1.4 MatrixType() [3/5]

```
void MatrixType (
            std::string fname,
            unsigned char & mtype,
            unsigned char & ctype,
            unsigned char & endianness )
```

Auxiliary funcions to check characteristics of matrix stored in binary file looking only at its header. Matrix in file is NOT loaded into memory.

Parameters

in	*fname*	Name of the binary file
out	*mtype*	Returns matrix type (full,sparse,symmetric)
out	*ctype*	Returns matrix data type
out	*endianness*	Returns endianness (big,little) of the data stored in the matrix
out	*nrows*	Returns the number of rows
out	*ncols*	Returns the number of columns
out	*mdinf*	Returns the signal for the presence of metadata

5.7.1.5 MatrixType() [4/5]

```
void MatrixType (
            std::string fname,
            unsigned char & mtype,
            unsigned char & ctype,
            unsigned char & endianness,
            unsigned char & mdinf )
```

Auxiliary funcions to check characteristics of matrix stored in binary file looking only at its header. Matrix in file is NOT loaded into memory.

Parameters

in	*fname*	Name of the binary file
out	*mtype*	Returns matrix type (full,sparse,symmetric)

Parameters

out	*ctype*	Returns matrix data type
out	*endianness*	Returns endianness (big,little) of the data stored in the matrix
out	*nrows*	Returns the number of rows
out	*ncols*	Returns the number of columns
out	*mdinf*	Returns the signal for the presence of metadata

5.7.1.6 MatrixType() [5/5]

```
void MatrixType (
            std::string fname,
            unsigned char & mtype,
            unsigned char & ctype,
            unsigned char & endianness,
            unsigned char & mdinf,
            indextype & nrows,
            indextype & ncols )
```

Auxiliary funcions to check characteristics of matrix stored in binary file looking only at its header. Matrix in file is NOT loaded into memory.

Parameters

in	*fname*	Name of the binary file
out	*mtype*	Returns matrix type (full,sparse,symmetric)
out	*ctype*	Returns matrix data type
out	*endianness*	Returns endianness (big,little) of the data stored in the matrix
out	*nrows*	Returns the number of rows
out	*ncols*	Returns the number of columns
out	*mdinf*	Returns the signal for the presence of metadata

5.8 src/headers/matmetadata.h File Reference

```
#include "fullmatrix.h"
#include "sparsematrix.h"
#include "symmetricmatrix.h"
#include <cmath>
```

Functions

- std::vector< std::string > JGetRowNames (std::string fname)
- std::vector< std::string > JGetColNames (std::string fname)

5.8.1 Function Documentation

5.8.1.1 JGetColNames()

```
std::vector<std::string> JGetColNames (
            std::string fname )
```

Function to get the names of the columns of the JMatrix stored in a binary file
The matrix is not loaded into memory

Parameters

in	*fname*	Name of the binary file containing the matrix

Returns

A vector of strings with the column names

5.8.1.2 JGetRowNames()

```
std::vector<std::string> JGetRowNames (
            std::string fname )
```

Function to get the names of the rows of the JMatrix stored in a binary file
The matrix is not loaded into memory

Parameters

in	*fname*	Name of the binary file containing the matrix

Returns

A vector of strings with the row names

5.9 src/headers/memhelper.h File Reference

```
#include <iostream>
```

Functions

- void GetAvailableMemAndSwap (unsigned long &avmem, unsigned long &avswap)
- void MemoryWarnings (unsigned long nr, unsigned long nc, int s)
- void MemoryWarnings (unsigned long nr, int s)

5.9.1 Function Documentation

5.9.1.1 GetAvailableMemAndSwap()

```
void GetAvailableMemAndSwap (
            unsigned long & avmem,
            unsigned long & avswap )
```

Finds how much memory is available and how much swap at call time.

Parameters

out	*avmem*	Available memory in bytes
out	*avswap*	Available swap in bytes ========== WARNING: currently this funcion work only on Linux systems. Calling in other systems simply sets the variables to 0 ==========

5.9.1.2 MemoryWarnings() [1/2]

```
void MemoryWarnings (
            unsigned long nr,
            int s )
```

Shows memory warnings if the memory needed to declare a SymmetricMatrix is above 75% of available memory or directly shows an error and stops the program is it is avobe the available memory
The idea is to call this function before going into an scope where a SymmetricMatrix will be declared if you think it might be too large, or if you don't know its size in advance

Parameters

in	*nr*	Number of rows of the prospective SymmetricMatrix (obviously, it will be square)
in	*s*	Size in bytes of each element (normally obtained with sizeof(Type)) ========== WARNING: currently this funcion work only on Linux systems. Calling in other systems simply sets the variables to 0 ==========

5.9.1.3 MemoryWarnings() [2/2]

```
void MemoryWarnings (
            unsigned long nr,
```

```
unsigned long nc,
int s )
```

Shows memory warnings if the memory needed to declare a FullMatrix is above 75% of available memory or directly shows an error and stops the program is it is avobe the available memory
The idea is to call this function before going into an scope where a FullMatrix will be declared if you think it might be too large, or if you don't know its size in advance

Parameters

in	*nr*	Number of rows of the prospective FullMatrix
in	*nc*	Number of columns of the prospective FullMatrix
in	*s*	Size in bytes of each element (normally obtained with sizeof(Type)) ========== WARNING: currently this funcion work only on Linux systems. Calling in other systems simply sets the variables to 0 ==========

5.10 src/headers/sparsematrix.h File Reference

```
#include "jmatrix.h"
#include <algorithm>
```

Classes

- class SparseMatrix< T >

Enumerations

- enum **TrMark** { **transpose** =0 }

Functions

- template<typename T >
  void **sort_indexes_and_value** (const std::vector< T > &v, std::vector< size_t > &idx, std::vector< indextype > &idv)

5.11 src/headers/symmetricmatrix.h File Reference

```
#include "jmatrix.h"
#include "memhelper.h"
```

Classes

- class SymmetricMatrix< T >

Index

~FullMatrix
FullMatrix< T >, 10
~SparseMatrix
SparseMatrix< T >, 25
~SymmetricMatrix
SymmetricMatrix< T >, 34

apitocommands.h
JCsvDump, 42
JCsvToJMat, 43
JGetColNames, 43
JGetNameCol, 43
JGetNameRow, 44
JGetNamesCol, 44
JGetNamesRow, 44
JGetNumCol, 45
JGetNumRow, 45
JGetNumsCol, 45
JGetNumsRow, 46
JGetRowNames, 46
JGetSubDiag, 46
JMatInfo, 47
JSetColNames, 47
JSetRowColNames, 47
JSetRowNames, 48

BIGEND
jmatrix.h, 56
BLOCK_MARK
jmatrix.h, 56
BLOCKSEP
jmatrix.h, 56
BLOCKSEP_LEN
jmatrix.h, 56

COL_NAMES
jmatrix.h, 56
COMMENT
jmatrix.h, 57
COMMENT_SIZE
jmatrix.h, 57

DataTypeName
jmatrix.h, 53
DEBJM
debugpar.h, 50
debugpar.h
DEBJM, 50
JMatrixSetDebug, 49
JMatrixStop, 49
JMatrixWarning, 49
NODEBUG, 50
DTYPE
jmatrix.h, 57

ERROR_READING_COL_NAMES
jmatrix.h, 57
ERROR_READING_ROW_NAMES
jmatrix.h, 57
ERROR_READING_SEP_MARK
jmatrix.h, 57
ERROR_READING_STRINGS
jmatrix.h, 57

FTYPE
jmatrix.h, 58
FullMatrix
FullMatrix< T >, 8–10
FullMatrix< T >, 7
~FullMatrix, 10
FullMatrix, 8–10
Get, 10
GetFullRow, 11
GetMarksOfFullRow, 11
GetRow, 12
GetUsedMemoryMB, 12
operator!=, 12
operator=, 12
Resize, 13
SelfColNorm, 13
SelfRowNorm, 14
Set, 14
WriteBin, 14
WriteCsv, 15

Get
FullMatrix< T >, 10
SparseMatrix< T >, 26
SymmetricMatrix< T >, 34
GetAvailableMemAndSwap
memhelper.h, 66
GetColNames
JMatrix< T >, 18
GetComment
JMatrix< T >, 18
GetFileSize
jmatrix.h, 54
GetFullRow
FullMatrix< T >, 11
GetMarksOfFullRow

FullMatrix< T >, 11
GetMarksOfSparseRow
SparseMatrix< T >, 26
GetNCols
JMatrix< T >, 19
GetNRows
JMatrix< T >, 19
GetRow
FullMatrix< T >, 12
SparseMatrix< T >, 27
GetRowNames
JMatrix< T >, 19
GetRowSum
SymmetricMatrix< T >, 34
GetSparseRow
SparseMatrix< T >, 27
GetUsedMemoryMB
FullMatrix< T >, 12
SparseMatrix< T >, 27
SymmetricMatrix< T >, 35

HEADER_SIZE
jmatrix.h, 58

indextype
indextype.h, 50
indextype.h
indextype, 50

JCsvDump
apitocommands.h, 42
JCsvToJMat
apitocommands.h, 43
JGetColNames
apitocommands.h, 43
matmetadata.h, 65
JGetNameCol
apitocommands.h, 43
JGetNameRow
apitocommands.h, 44
JGetNamesCol
apitocommands.h, 44
JGetNamesRow
apitocommands.h, 44
JGetNumCol
apitocommands.h, 45
JGetNumRow
apitocommands.h, 45
JGetNumsCol
apitocommands.h, 45
JGetNumsRow
apitocommands.h, 46
JGetRowNames
apitocommands.h, 46
matmetadata.h, 65
JGetSubDiag
apitocommands.h, 46
jmat.cpp
main, 39
JMatInfo
apitocommands.h, 47
matinfo.h, 61
JMatrix
JMatrix< T >, 16–18
JMatrix< T >, 15
GetColNames, 18
GetComment, 18
GetNCols, 19
GetNRows, 19
GetRowNames, 19
JMatrix, 16–18
operator!=, 19
operator=, 20
Resize, 20
SetColNames, 20
SetComment, 21
SetRowNames, 21
WriteBin, 21
WriteCsv, 22
jmatrix.h
BIGEND, 56
BLOCK_MARK, 56
BLOCKSEP, 56
BLOCKSEP_LEN, 56
COL_NAMES, 56
COMMENT, 57
COMMENT_SIZE, 57
DataTypeName, 53
DTYPE, 57
ERROR_READING_COL_NAMES, 57
ERROR_READING_ROW_NAMES, 57
ERROR_READING_SEP_MARK, 57
ERROR_READING_STRINGS, 57
FTYPE, 58
GetFileSize, 54
HEADER_SIZE, 58
LDTYPE, 58
LITEND, 58
MatrixTypeName, 54
MAX_LEN_NAME, 58
MetadataInfo, 54
MTYPEFULL, 58
MTYPENOTYPE, 58
MTYPESPARSE, 59
MTYPESYMMETRIC, 59
NO_METADATA, 59
NOTYPE, 59
PositionsInFile, 55
READ_OK, 59
ROW_NAMES, 59
SCTYPE, 59
SITYPE, 60
SizeOfType, 55
SLLTYPE, 60
SLTYPE, 60
SSTYPE, 60
ThisMachineEndianness, 56

UCTYPE, 60
UITYPE, 60
ULLTYPE, 60
ULTYPE, 60
USTYPE, 61
JMatrixSetDebug
debugpar.h, 49
JMatrixStop
debugpar.h, 49
JMatrixWarning
debugpar.h, 49
JSetColNames
apitocommands.h, 47
JSetRowColNames
apitocommands.h, 47
JSetRowNames
apitocommands.h, 48

LDTYPE
jmatrix.h, 58
LITEND
jmatrix.h, 58

main
jmat.cpp, 39
matinfo.h
JMatInfo, 61
MatrixType, 62–64
matmetadata.h
JGetColNames, 65
JGetRowNames, 65
MatrixType
matinfo.h, 62–64
MatrixTypeName
jmatrix.h, 54
MAX_LEN_NAME
jmatrix.h, 58
memhelper.h
GetAvailableMemAndSwap, 66
MemoryWarnings, 66
MemoryWarnings
memhelper.h, 66
MetadataInfo
jmatrix.h, 54
MTYPEFULL
jmatrix.h, 58
MTYPENOTYPE
jmatrix.h, 58
MTYPESPARSE
jmatrix.h, 59
MTYPESYMMETRIC
jmatrix.h, 59

NO_METADATA
jmatrix.h, 59
NODEBUG
debugpar.h, 50
NOTYPE
jmatrix.h, 59

operator!=
FullMatrix< T >, 12
JMatrix< T >, 19
SparseMatrix< T >, 28
operator=
FullMatrix< T >, 12
JMatrix< T >, 20
SparseMatrix< T >, 28
SymmetricMatrix< T >, 35

PositionsInFile
jmatrix.h, 55

READ_OK
jmatrix.h, 59
Resize
FullMatrix< T >, 13
JMatrix< T >, 20
SparseMatrix< T >, 28
SymmetricMatrix< T >, 35
ROW_NAMES
jmatrix.h, 59

SCTYPE
jmatrix.h, 59
SelfColNorm
FullMatrix< T >, 13
SparseMatrix< T >, 29
SelfRowNorm
FullMatrix< T >, 14
SparseMatrix< T >, 29
Set
FullMatrix< T >, 14
SparseMatrix< T >, 29
SymmetricMatrix< T >, 36
SetColNames
JMatrix< T >, 20
SetComment
JMatrix< T >, 21
SetRow
SparseMatrix< T >, 30
SetRowNames
JMatrix< T >, 21
SITYPE
jmatrix.h, 60
SizeOfType
jmatrix.h, 55
SLLTYPE
jmatrix.h, 60
SLTYPE
jmatrix.h, 60
SparseMatrix
SparseMatrix< T >, 24, 25
SparseMatrix< T >, 23
~SparseMatrix, 25
Get, 26
GetMarksOfSparseRow, 26
GetRow, 27
GetSparseRow, 27

GetUsedMemoryMB, 27
operator!=, 28
operator=, 28
Resize, 28
SelfColNorm, 29
SelfRowNorm, 29
Set, 29
SetRow, 30
SparseMatrix, 24, 25
WriteBin, 30
WriteCsv, 30
src/examples/jmat.cpp, 39
src/headers/apitocommands.h, 42
src/headers/debugpar.h, 48
src/headers/fullmatrix.h, 50
src/headers/indextype.h, 50
src/headers/jmatrix.h, 51
src/headers/matinfo.h, 61
src/headers/matmetadata.h, 64
src/headers/memhelper.h, 65
src/headers/sparsematrix.h, 67
src/headers/symmetricmatrix.h, 67
SSTYPE
jmatrix.h, 60
SymmetricMatrix
SymmetricMatrix< T >, 32, 33
SymmetricMatrix< T >, 31
~SymmetricMatrix, 34
Get, 34
GetRowSum, 34
GetUsedMemoryMB, 35
operator=, 35
Resize, 35
Set, 36
SymmetricMatrix, 32, 33
TestDistDisMat, 36
WriteBin, 36
WriteCsv, 37

TestDistDisMat
SymmetricMatrix< T >, 36
ThisMachineEndianness
jmatrix.h, 56

UCTYPE
jmatrix.h, 60
UITYPE
jmatrix.h, 60
ULLTYPE
jmatrix.h, 60
ULTYPE
jmatrix.h, 60
USTYPE
jmatrix.h, 61

WriteBin
FullMatrix< T >, 14
JMatrix< T >, 21
SparseMatrix< T >, 30
SymmetricMatrix< T >, 36
WriteCsv
FullMatrix< T >, 15
JMatrix< T >, 22
SparseMatrix< T >, 30
SymmetricMatrix< T >, 37